

实验报告Lab2

231250136 刘怡然

代码结构设计

1. 静态作用域分析 (src/executor/AccessLinkBuilder.cj)

在 src/executor 中添加了 AccessLinkBuilder 类。

- 功能：
 - 负责在解释器运行前对 AST 进行一遍静态扫描。
 - 构建每个 AST 节点（主要是 Block 和函数定义）的静态父级作用域（Access Link）模板。
 - 也为了实现对于函数body空块的识别

- 核心函数：

```
public func buildAccessLink(program: Program): Map<Node,
ActivationRecords> { // 返回每个block及其对应的活动记录模板
    var globalRecords: ActivationRecords =
BlockActivationRecords("Global", None);

    addLibFunc2GlobalRecords(globalRecords);

    accessLinkMap.add(program, globalRecords);

    for (decl in program.decls) {
        match (decl) {
            case mainDecl: MainDecl =>
                var mainRecord: Record = Record(mainDecl.identifier,
Value.from(MainValue(mainDecl)), None,
                mainDecl.keyword, mainDecl);
                globalRecords.addRecord(mainDecl.identifier.value,
mainRecord, mainDecl);
                buildMainAccessLink(mainDecl, globalRecords);
            case funcDecl: FuncDecl =>
                var funcRecord: Record = Record(funcDecl.identifier,
Value.from(FuncValue(funcDecl, None)), None,
                funcDecl.keyword, funcDecl);
                globalRecords.addRecord(funcDecl.identifier.value,
funcRecord, funcDecl);
                buildFuncAccessLink(funcDecl, globalRecords);
            case varDecl: VarDecl => () // 全局变量在Evaluator中做
            case funcParam: FuncParam => ()
```

```

        case _ => throw UnhandledDeclInAccessLinkBuilder(decl);
    }
}

return accessLinkMap;
}

```

2. 修正 (src/executor/ActivationRecords.cj)

修正了 ActivationRecords 及其子类 FuncActivationRecords 和 BlockActivationRecords。

- 变量查找功能：
 - 能区分 Control Link (动态链) 和 Access Link (静态链)。Control Link 指向调用者，用于函数返回；Access Link 指向定义时的环境，用于变量查找。
 - 在 getStaticRecord 和 setStaticRecord 中增加了 isNonLocal 标志位及递归传递逻辑，为了实现 **内层函数不允许访问外层函数的可变非局部变量** 的语义检查
- 传参：
 - 在 FuncActivationRecords 中实现了 assignParams，并在绑定形参时强制将其 keyword 设为 LET，确保形参在函数体内不可被重新赋值。

3. 添加函数值的表示 (src/executor/Value.cj)

在 Value 枚举中加入了函数的运行时表示：

- VFunction(FuncValue):
 - FuncValue 类中包含 FuncDecl（函数定义）和 capturedEnv（捕获的闭包环境）。

```

public class FuncValue <: ToString {
    public var funcDecl: FuncDecl
    public var capturedEnv: ?ActivationRecords

    public init(_funcDecl: FuncDecl, _env: ?ActivationRecords) {
        this.funcDecl = _funcDecl
        this.capturedEnv = _env
    }

    public override func toString(): String {
        "func"
    }
}

```

- VMain(MainValue): 用于特殊处理 main 函数。

4. Evaluator中的实现 (src/executor/Evaluator.cj)

- 构建全局函数和全局变量：
 - 先 visitProgram 注册所有全局函数)。然后再按顺序初始化全局变量。
- 作用域链式嵌套：
 - 在 visit(VarDecl) 中，每定义一个变量都会创建一个新的嵌套 BlockActivationRecords，替代旧的栈顶记录，这里是因为比如如下样例：

```
int x = 100
main() {
    // 1. 定义函数 f
    func f() {
        println(x)
    }

    var x = 10
}
```

函数 f 不应该捕获到 x 所以查找的时候不能仅仅在上一层Block中的作用域找，变量的声明应该有顺序

- 函数调用 (visit(CallExpr)):
 - 实现了函数参数类型检查（当前执行到的参数不匹配就立刻抛异常）
 - 实现了返回值类型检查
 - 对于 return 语句和lab1处理 while break 类似，return 直接抛出异常，函数在外面 catch

5. 变量记录 (src/executor/Record.cj)

- 增强了 updateValue 逻辑：
 - 使得 FUNC 类型的不可变，从而实现禁止对函数名赋值。
 - 函数类型检查封装在了Evaluator里面，下次应该改到封装到 updateValue 中。

我认为的亮点

1. 函数类型的兼容

- 在原本只支持lab1类型的 Record 中增加对于函数类型的支持而尽量不修改lab1实现的visit代码，很符合函数是一等公民的概念，对函数的检查也是递归的，所以函数类型及其嵌套嵌套对于visit来说也是透明的，和基本类型一样。

2. 实现返回值

- 使用 `ReturnException` 机制来处理 `return` 语句，能够从深层嵌套的语句块中直接跳出并携带返回值，直到被 `visit(CallExpr)` 捕获，同时也会恢复调用栈。

遇到的问题 and 解决方案

1. 问题：一开始的思路是通过一个 `AccessLinkBuilder` 类将静态的作用域维护好（构建好 `AccessLink`）同时创建好要使用的活动记录(`ActivationRecords`)模板，然后在运行时直接把模板拿来使用
 - **现象和原因**：在函数递归调用的时候可能会多次创建相同函数的栈帧，而这些栈帧同时会修改同一份`ActivationRecords`，就会出现冲突导致结果不对
 - **解决方案**：需要记录闭包捕获到的变量，然后利用`AccessLinkBuilder`的逻辑使得 `accessLink` 指向闭包捕获到的变量，而不是指向包裹这个闭包的活动记录 (`ActivationRecords`)，然后在运行的时候再动态创建对应的活动记录，使得每个栈帧的活动记录隔离，避免冲突
2. 问题：无法确保嵌套函数忽略其定义时尚未声明的外层局部变量
 - **现象和原因**：一开始的设计是为每一个`block`或`function`创建一个活动记录，然后在他们的活动记录中存放对应作用域的变量（通过`map`存放，没有顺序），然后在变量查找时，沿着 `accessLink`在活动记录中找变量的定义。由于没有顺序，就会导致无法确保嵌套函数忽略其定义时尚未声明的外层局部变量。
 - **解决方案**：这里的解决方案虽然能解决问题，但是确实不好（下次lab前会尽量修复），理想中的方式是活动记录中的变量规定顺序，但是由于代码设计的问题不好实现。所以就采取了一种有点投机取巧的方式：对每一个声明的变量都用一个活动记录（`ActivationRecords`）包裹，然后前面声明的变量的活动记录会指向（`controlLink`）下一个变量声明的活动记录，然后再加上对于同一个静态作用域防止同名变量定义的补丁。